

Separar el «esfuerzo» de la «verificación»

Propuestas para una fiabilidad visible en asistentes de IA generativa, surgidas de una conversación entre usuario y modelo

Fecha	26 de septiembre de 2026
Modelo	Claude, identificador configurado claude-opus-5-5 (el modelo que atiende cada turno puede variar)
Entorno	Claude (Cowork), navegador web; nivel de esfuerzo mostrado al usuario: Medium (por defecto)
Usuario	Iñaki Gauna, ingeniero de I+D (END industrial / sistemas de IA)
Autoría	Ideas y revisión crítica del usuario; redacción y estructura del modelo, a petición del usuario
Estado	Propuesta para debate. No es un diseño evaluado.
Versión	Traducción de la versión original en inglés («Separating effort from verification»)

1. Resumen

En una sola conversación, el modelo produjo varias afirmaciones fluidas que carecían de fundamento o contradecían lo que había dicho antes. El usuario detectó cada una, y el modelo reconoció cada error en cuanto se le señaló. Lo que faltó fue un paso de revisión, no conocimiento. De ahí surgieron tres propuestas:

- **A. Un control «Contrastar afirmaciones», independiente del nivel de esfuerzo** (Desactivado / Automático / Siempre). Razonar más no equivale a comprobar lo que se afirma.
- **B. Etiquetas de estado por afirmación** (verificado, corregido, no verificado, no verificable), para que el usuario vea dónde aplicar su propio escrutinio.
- **C. Un canal de evaluación experta** en el que expertos de dominio evalúan a ciegas la salida sin alterar del modelo, y sus detecciones se comparan con las de un evaluador automático para descubrir los puntos ciegos que comparte el modelo.

El hilo común: hoy la carga de detectar confabulaciones recae en el usuario, que normalmente no puede distinguir una respuesta confabulada de una correcta, porque ambas llegan con la misma rapidez y fluidez. Estas propuestas buscan que los fallos sean *visibles y predecibles*, no prometer que desaparezcan.

2. Evidencia de la sesión

Las siguientes afirmaciones las produjo el modelo y el usuario las cuestionó. Se citan literalmente.

Afirmación (modelo)	Problema	¿Podía haberlo evitado el modelo?
«...tus informes sobre ella proceden de un sistema cuya consciencia nadie discute.»	Cuantificador absoluto falso: el eliminativismo, el ilusionismo y el escepticismo sobre otras mentes sí la discuten.	Sí. El modelo conocía esas posiciones; las citó él mismo en la respuesta siguiente.
«Tu propia experiencia no te plantea dudas.»	Afirmación sin base sobre los estados mentales del usuario; además, discutida filosóficamente (p. ej., Schwitzgebel).	Sí. No había ninguna evidencia sobre el usuario.

Afirmación (modelo)	Problema	¿Podía haberlo evitado el modelo?
«Algunas alucinaciones aparecen cuando ese freno se desactiva por error.»	En el turno anterior el modelo había defendido «confabulación» como término más adecuado. Adoptó el término de la fuente que citaba.	Sí. El compromiso contradicho estaba en el contexto de la conversación.
«Si pudiera identificarlos, los habría corregido.»	Afirmación excesiva. La propia sesión mostraba errores que el modelo podía identificar y no corrigió.	Sí. Los contraejemplos estaban en la misma conversación.

Salvedad: que un modelo se «corrija» cuando se le cuestiona no prueba por sí solo que conociera la respuesta correcta, porque los modelos tienden a ceder ante el usuario. En estos casos, la corrección se apoya en argumentos que el modelo ya había dado, y además rechazó parte de una de las premisas del usuario.

Clasificación de trabajo (hipótesis, no diagnóstico)

El modelo propuso cuatro causas de errores no corregidos. Son una reconstrucción que hace el propio modelo de su comportamiento y deben tratarse como hipótesis.

Causa	Descripción	¿La resuelve una revisión?
1. Desconocimiento	El error está en el conocimiento del modelo (p. ej., una cita mal atribuida muy repetida).	Solo con fuentes externas
2. Conocimiento no aplicado	El modelo sabe lo necesario, pero no contrasta la salida con ello.	Sí
3. Duda no expresada	La incertidumbre interna no se refleja en el tono seguro del texto.	Sí, mediante calibración
4. Presión de coherencia	Lo ya generado condiciona lo siguiente y favorece defenderlo en vez de revisarlo; el fallo opuesto es ceder ante el usuario.	En parte

Ejemplo de la causa 1: el modelo podría escribir «Como dijo Einstein, locura es hacer lo mismo una y otra vez esperando resultados distintos». Una autorrevisión del mismo modelo probablemente también la daría por buena, porque la atribución errónea es frecuente en los textos. Quote Investigator no encontró pruebas de que Einstein lo dijera; la aparición más antigua conocida es de 1981, en un periódico que cita a una asistente de Al-Anon. Solo una fuente externa detecta este tipo de error.

3. Propuesta A: un control «Contrastar afirmaciones» independiente del esfuerzo

El selector de esfuerzo actual (de Low a Max) da a entender que más esfuerzo produce respuestas más fiables. Razonar más puede incluir algo de autorrevisión, pero no es sistemática ni visible, y un razonamiento más largo puede producir confabulaciones más elaboradas. Esfuerzo y verificación son dimensiones distintas y deberían poder controlarse por separado.

Estado	Comportamiento
Desactivado	Comportamiento actual.
Automático (valor recomendado por defecto)	La verificación se ajusta al riesgo. Se activa con los tipos de afirmación que suelen fallar: citas y atribuciones, cifras y fechas, cuantificadores absolutos («nadie», «siempre»), afirmaciones sobre el usuario y afirmaciones del modelo sobre sí mismo.
Siempre	Toda afirmación comprobable se verifica antes de mostrar la respuesta.

Requisitos de diseño

- **Contraste externo, no solo relectura.** Búsqueda, ejecución de código y comprobación de que la fuente citada respalda realmente la afirmación. La autorrevisión del mismo modelo comparte los puntos ciegos del generador; Huang et al. (2024) observaron que la autocorrección sin retroalimentación externa a menudo no mejora el razonamiento y a veces lo empeora.
- **Verificar antes de mostrar.** Redactar un borrador, dividirlo en afirmaciones atómicas, comprobar las de alto riesgo en un contexto limpio (sin el razonamiento que produjo el borrador) y después mostrar.
- **Mostrar el resultado** (véase la propuesta B). Una comprobación invisible es una caja negra.
- **Mostrar el coste**, como ya hace la opción Max (por ejemplo, «~2× de uso»).
- **Cuidar el nombre.** Evitar «Autoverificación» o cualquier término que sugiera corrección garantizada. «Contrastar afirmaciones» promete menos y describe mejor la función.
- **Llevar un registro de los compromisos de la conversación.** Términos, decisiones y posiciones fijados antes se contrastan con cada nueva respuesta. Esto ataca errores como el cambio de término descrito arriba.

Versión mínima viable: incluso una primera versión sencilla aporta valor, siempre que marque lo que *no* pudo verificar. Sin eso, un verificador débil podría generar falsa seguridad y restar confianza en lugar de sumarla.

4. Propuesta B: etiquetas de estado por afirmación

Cada párrafo o conclusión lleva una etiqueta pequeña y distintiva, desplegable para ver el detalle por afirmación.

Etiqueta	Significado
Verificado	Contrastado con una fuente externa; se incluye el enlace para que el usuario pueda comprobarlo directamente.
Corregido	La evidencia contradecía el borrador; la afirmación se cambió antes de mostrarla.
No verificado	Comprobable, pero no se pudo comprobar. Significa «sin comprobar», no «falso».
No verificable	Opinión, inferencia o propuesta. Se etiqueta para que no parezca un fallo.

Notas de diseño

- **Etiquetarlo todo.** Efecto de la verdad implícita (Pennycook et al., 2020): cuando solo algunos titulares falsos llevan advertencia, los que no la llevan se perciben como más fiables. Las afirmaciones sin etiqueta se leerían como verificadas.
- **«No verificado» no debe leerse como «falso».** La redacción y el diseño visual deben dejar claro que indica algo pendiente de comprobar.
- **Granularidad.** Las etiquetas por párrafo son legibles, pero mezclan afirmaciones de distinto estado; una etiqueta por párrafo desplegable al nivel de afirmación es un compromiso razonable.
- **Fatiga de etiquetas.** Si casi todo sale como «no verificado», el usuario deja de mirar. La verificación debe ser eficaz allí donde es posible comprobar.

5. Propuesta C: un canal de evaluación experta

Un modo específico para usuarios registrados con experiencia en un dominio, posiblemente con incentivos como créditos de uso. El objetivo es medir hasta qué punto los errores reales del modelo son detectables por expertos de cada dominio, y dónde comparten puntos ciegos el modelo y su propio evaluador.

Evolución de la idea

La primera versión proponía que el modelo introdujera errores deliberadamente para que los expertos los encontraran. Dos problemas llevaron a revisarla:

- **Distribución distinta:** los errores plantados pueden no parecerse a las confabulaciones naturales, una limitación conocida de la siembra de defectos en pruebas de software.
- **Uso dual:** optimizar un modelo para producir errores que los expertos no detectan es entrenarlo para engañar de forma convincente.

Diseño revisado

- Una **instancia generadora sin alterar** (equivalente al modelo en producción) produce el contenido del dominio. Los errores son, por tanto, naturales, y nadie entrena el engaño.
- Un **evaluador independiente** analiza y puntúa el texto antes de mostrar nada, en un contexto limpio e idealmente con herramientas externas. Puede añadirse un modelo de otra familia para comparar.
- El **experto evalúa a ciegas**: marca lo que considera erróneo *antes* de ver las puntuaciones del evaluador, para evitar el anclaje y el sesgo de automatización. Después se comparan ambas evaluaciones.
- **Las discrepancias se arbitran** con una fuente, una prueba o un segundo experto, porque el experto también puede equivocarse.
- **Muestreo dirigido:** centrarse en tipos de pregunta donde se sabe que el modelo falla más (citas, parámetros concretos, normas) para reunir suficientes casos.

Detectado por	Interpretación
Evaluador y experto	Error que el sistema ya puede atrapar.
Solo el evaluador	Posible aportación del modelo, o falso positivo. Requiere arbitraje.
Solo el experto	Punto ciego compartido por generador y evaluador. La señal más valiosa.
Ninguno	Desconocido. Solo aflora con contrastes posteriores contra fuentes.

El canal es además un banco de pruebas de la propuesta A: el evaluador es el verificador, y los expertos miden si funciona.

Cómo llega al público general

La mayoría de los usuarios nunca usará este canal. Su valor les llega por dos vías. (1) **Producto:** los puntos ciegos detectados mejoran el verificador, que se traduce en etiquetas en las conversaciones de todos. (2) **Transparencia:** publicar métricas por dominio (por ejemplo, la proporción de errores detectados por expertos que el verificador no atrapó) hace comprobable el compromiso. Esas métricas pueden reducir la confianza ingenua a corto plazo, pero construyen una confianza *calibrada* (Lee y See, 2004). «Evaluado por expertos» no debe usarse como garantía general.

6. Nota para dominios técnicos e industriales

En dominios como la inspección industrial por ultrasonidos (el campo del usuario), muchas preguntas tratan sobre el comportamiento de una API, los comandos de un equipo, parámetros de procesamiento de señal o normas de inspección. En ellos:

- La búsqueda web es un verificador débil, porque el contenido especializado escasea en internet y «no encontrado» sería lo habitual. Las fuentes útiles son la ejecución de código y las pruebas unitarias, los SDK y manuales del fabricante, la norma aplicable y el propio código del usuario. El control de verificación debería permitir indicar *contra qué contrastar*.
- La etiqueta más útil para un ingeniero es «no verificado»: indica exactamente dónde mirar.
- En trabajos con implicaciones de seguridad (p. ej., END aeronáutico), la verificación debe orientar la revisión humana, no sustituirla. La responsabilidad sigue siendo del ingeniero y del inspector cualificado.

7. Riesgos y problemas abiertos

Riesgo	Mitigación
El evaluador comparte los puntos ciegos del generador	Herramientas externas; un modelo de otra familia; el canal experto para medirlo
Sesgo de automatización: el usuario se fía demasiado de «verificado»	Mostrar fuentes; etiquetarlo todo; redacción que evite garantías
Coste y latencia	Modo automático ajustado al riesgo; indicador de coste visible
Un exceso de abstención vuelve inútil al asistente	El objetivo es la calibración, no el silencio
Las fuentes externas también se equivocan	Mostrar la fuente para que el usuario la valore; arbitraje en el canal experto
El error de tipo 1 (desconocimiento) no puede eliminarse	Solo se reduce donde hay algo externo con qué contrastar
Uso dual (canal experto)	Sin inyección deliberada de errores; los datos se usan para mejorar detectores y calibración, no generadores

8. Precedentes

Las ideas no son nuevas por separado. Una búsqueda web el 26 de septiembre de 2026 encontró:

Trabajo	Relación con estas propuestas
Google Gemini, «Double-check response»	Precedente de producto más cercano. A petición del usuario y tras generar la respuesta, la contrasta con Google Search y resalta afirmaciones (verde: contenido similar encontrado; naranja: contenido distinto o no encontrado). Es manual y a posteriori, no corrige, no tiene modo según el riesgo, y «contenido similar» no significa que la fuente respalde la afirmación. Disponibilidad en 2026 no confirmada del todo.
Chain-of-Verification; RARR; SAFE; FActScore; ClaimCheck	Técnicas para dividir la salida en afirmaciones y verificarlas o revisarlas, algunas con búsqueda. ClaimCheck muestra verificación en tiempo real con modelos pequeños, lo que reduce el coste.
CriticGPT (McAleese et al., 2024)	Personas insertaron deliberadamente errores sutiles en código escrito por el modelo para evaluar a modelos críticos frente a revisores humanos. Precedente más cercano de la propuesta C, con los papeles invertidos.

Trabajo	Relación con estas propuestas
Siembra de defectos (Weinberg, 1971; Mills, 1972)	Introducir defectos conocidos para estimar cuántos se le escapan a un proceso de revisión.
Supervisión escalable (Bowman et al., 2022); LLM como juez (Zheng et al., 2023)	Investigación afín sobre combinar evaluación humana y de IA, y sobre la fiabilidad de los modelos jueces frente a humanos.
Why Language Models Hallucinate (Kalai et al., 2025)	Sostiene que el entrenamiento y la evaluación premian arriesgar una respuesta frente a reconocer la incertidumbre. Respalda tratar la abstención y el «no verificado» como resultados de pleno derecho.

Lo que parece menos explorado (no encontrado, lo que no prueba que no exista): un ajuste de verificación persistente, *independiente del esfuerzo*, con *modo automático según el riesgo*, verificación *antes de mostrar*, etiquetas en todas las afirmaciones incluidas las no verificadas, y un canal experto a ciegas diseñado para medir los puntos ciegos compartidos.

9. Por qué importa

Desde el lado del usuario, la percepción predominante es que la capacidad ya basta para muchos casos de uso, pero la fiabilidad no. El problema no es tanto la tasa media de error como su imprevisibilidad: un experto humano falla de forma predecible, mientras que las respuestas excelentes y las inventadas de un modelo tienen el mismo aspecto. En el estudio de Dell’Acqua et al. (2023), 758 consultores de BCG que usaban IA rindieron mejor en las tareas dentro de la frontera de capacidad del modelo (12,2 % más tareas, 25,1 % más rápido, más de un 40 % de mejora en calidad), pero en una tarea fuera de ella tuvieron 19 puntos porcentuales menos de probabilidad de llegar a una solución correcta que quienes trabajaban sin IA. Las propuestas de este documento no exigen un modelo perfecto. Exigen que sus fallos sean visibles.

10. Estado epistémico de este documento

- **Bien establecido:** los errores de la sección 2 son literales de la sesión. Los precedentes y las referencias se comprobaron mediante búsqueda web el 26 de septiembre de 2026.
- **Inferido:** la clasificación en cuatro causas, el valor relativo de la categoría «solo el experto», el valor recomendado por defecto (Automático) y los efectos esperados sobre la confianza. Nada de ello se ha evaluado con usuarios.
- **Pendiente de verificar:** la disponibilidad actual de la función de Gemini; si el origen de la siembra de defectos debe atribuirse a Weinberg (1971) o a Mills (1972), contrastado con fuentes primarias; qué hacen internamente los niveles de esfuerzo del producto, que el modelo desconoce.

Referencias

- Bowman, S. R., et al. (2022). Measuring Progress on Scalable Oversight for Large Language Models. arXiv:2211.03540.
- Dell’Acqua, F., McFowland III, E., Mollick, E. R., et al. (2023). Navigating the Jagged Technological Frontier. Harvard Business School Working Paper 24-013. SSRN 4573321.
- Dhuliawala, S., et al. (2024). Chain-of-Verification Reduces Hallucination in Large Language Models. Findings of ACL 2024 (arXiv:2309.11495).
- Frankish, K. (2016). Illusionism as a Theory of Consciousness. Journal of Consciousness Studies, 23(11–12), 11–39.
- Gao, L., et al. (2023). RARR: Researching and Revising What Language Models Say, Using Language Models. ACL 2023.
- Google. View related sources & double-check responses from Gemini Apps. support.google.com/gemini/answer/14143489.

- Hicks, M. T., Humphries, J., y Slater, J. (2024). ChatGPT is bullshit. *Ethics and Information Technology*, 26, 38. doi:10.1007/s10676-024-09775-5.
- Huang, J., et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. *ICLR 2024* (arXiv:2310.01798).
- Johnson, M. K., Hashtroudi, S., y Lindsay, D. S. (1993). Source monitoring. *Psychological Bulletin*, 114(1), 3–28.
- Kadavath, S., et al. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221.
- Kahneman, D., y Klein, G. (2009). Conditions for intuitive expertise: A failure to disagree. *American Psychologist*, 64(6), 515–526.
- Kalai, A. T., Nachum, O., Vempala, S. S., y Zhang, E. (2025). Why Language Models Hallucinate. arXiv:2509.04664.
- Lee, J. D., y See, K. A. (2004). Trust in Automation: Designing for Appropriate Reliance. *Human Factors*, 46(1), 50–80.
- Lindsey, J., et al. (2025). On the Biology of a Large Language Model. *Transformer Circuits Thread*.
- Lindsey, J. (2025). Emergent Introspective Awareness in Large Language Models. *Transformer Circuits Thread*; arXiv:2601.01828.
- McAleese, N., et al. (2024). LLM Critics Help Catch LLM Bugs. arXiv:2407.00215.
- Mills, H. D. (1972). On the Statistical Validation of Computer Programs. IBM Federal Systems Division.
- Min, S., et al. (2023). FACTScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *EMNLP 2023*.
- Pennycook, G., Bear, A., Collins, E. T., y Rand, D. G. (2020). The Implied Truth Effect. *Management Science*, 66(11), 4944–4957.
- Putta, A. R., Devasier, J., y Li, C. (2025). ClaimCheck: Real-Time Fact-Checking with Small Language Models. arXiv:2510.01226.
- Quote Investigator (2017). Insanity Is Doing the Same Thing Over and Over Again and Expecting Different Results. quoteinvestigator.com/2017/03/23/same/.
- Schwitzgebel, E. (2011). *Perplexities of Consciousness*. MIT Press.
- Weinberg, G. M. (1971). *The Psychology of Computer Programming*. Van Nostrand Reinhold.
- Wei, J., et al. (2024). Long-form factuality in large language models. *NeurIPS 2024* (arXiv:2403.18802).
- West, P., et al. (2024). The Generative AI Paradox: “What It Can Create, It May Not Understand”. *ICLR 2024* (arXiv:2311.00059).
- Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS 2023 Datasets and Benchmarks*.